

Telecom RAG Chatbot Deployment Guide

Beginner-friendly, step-by-step documentation of the completed Render + Vercel deployment

Project: [ujoy100/telecom-rag-chatbot](https://github.com/ujoy100/telecom-rag-chatbot) | Completed: 7 October 2026

1. What We Deployed

The application has two separately deployed parts. The React/Vite frontend is hosted on Vercel. The FastAPI backend is hosted on Render. The backend performs retrieval from Chroma and calls the selected LLM. The frontend never stores the Groq or OpenAI API keys.

Item	Status
Frontend	Vercel - LIVE
Backend	Render - LIVE
Vector database	Chroma
Embeddings	OpenAI text-embedding-3-small
LLM choices	Groq Qwen and OpenAI
Retrieval evaluation	Recall@3 = 100% (10/10 test cases)

2. Final Production URLs

Frontend: <https://telecom-rag-chatbot.vercel.app>

Backend: <https://telecom-rag-chatbot-poqa.onrender.com>

Backend health check: <https://telecom-rag-chatbot-poqa.onrender.com/health>

Important: the Render free instance can spin down after inactivity. The first request after a quiet period can therefore take noticeably longer.

3. Architecture - What Happens When a User Asks a Question

1) The user opens the Vercel website and enters a question. 2) React sends the question and selected provider to the Render FastAPI /chat/stream endpoint. 3) The backend converts the question to an embedding using OpenAI. 4) Chroma searches the FAQ, resolved-ticket, and guide collections. 5) If the best result is sufficiently relevant, the retrieved context is sent to Groq or OpenAI. 6) The answer is streamed back to the browser with a source label. 7) If the question is outside the knowledge base, the application returns the fixed fallback response instead of inventing an answer.

4. Why the First Render Deployment Failed

The original project used sentence-transformers/all-MiniLM-L6-v2 through HuggingFaceEmbeddings. That required PyTorch and related local ML packages to be loaded inside the Render service. The free Render instance had 512 MB RAM, and startup exceeded that limit. Render reported an out-of-memory failure before the service could stay live.

The solution was to move embedding computation to the OpenAI Embeddings API. This removed the need to load the local SentenceTransformer/PyTorch model in the web service.

5. Step A - Remove Heavy Local Embedding Dependencies

We first removed torchvision and later removed the local HuggingFace/SentenceTransformer/PyTorch dependencies.

```
uv remove torchvision
grep -nE "torch|torchvision|sentence-transformers" pyproject.toml
```

After the code migration, the relevant project dependencies were:

```
dependencies = [
    "chromadb>=1.5.9",
    "fastapi>=0.142.2",
    "langchain>=1.4.3",
    "langchain-chroma>=1.1.0",
    "langchain-community>=0.4.2",
    "langchain-core>=1.6.6",
    "langchain-groq>=1.1.3",
    "langchain-openai>=1.6.7",
    "langchain-text-splitters>=1.1.3",
    "pandas>=3.0.6",
    "pypdf>=6.19.0",
    "python-dotenv>=1.2.4",
    "uvicorn[standard]>=0.54.0",
]
```

6. Step B - Create a Shared OpenAI Embedding Module

A shared backend/embeddings.py module was created so ingestion and retrieval use exactly the same embedding model.

```
from langchain_openai import OpenAIEmbeddings

EMBED_MODEL = "text-embedding-3-small"

def get_embeddings():
    """Return the embedding model shared by ingestion and retrieval."""
    return OpenAIEmbeddings(model=EMBED_MODEL)
```

The retriever, FAQ ingestion, ticket ingestion, PDF ingestion, and retrieval evaluation were changed to call get_embeddings() instead of HuggingFaceEmbeddings.

7. Step C - Verify No Old HuggingFace Embedding Code Remained

We searched all Python files under backend:

```
grep -rniE "HuggingFaceEmbeddings|langchain_huggingface|all-MiniLM|sentence-transformers"
backend --include="*.py"
```

The final command returned no matches, confirming the old embedding implementation was removed from backend Python code.

8. Step D - Rebuild the Chroma Vector Collections

Changing the embedding model means the existing vectors cannot simply be reused. The knowledge base must be embedded again with the new model.

Because the ingestion modules import backend.embeddings, they were run as Python modules from the repository root. Running python backend/ingest_faq.py caused a ModuleNotFoundError for backend; using -m fixed the import context.

```
uv run python -m backend.ingest_faq
uv run python -m backend.ingest_tickets
uv run python -m backend.ingest_pdf
```

Item	Status
FAQ collection	25 vectors stored
Tickets collection	19 vectors stored
Guides/PDF collection	23 vectors stored
Total	67 vectors

9. Step E - Environment Variable Check

OpenAIEmbeddings requires OPENAI_API_KEY. Locally, the key was kept in .env and never committed to Git. A safe check should only confirm whether the variable exists; it should never print the key itself.

```
uv run python -c "from dotenv import load_dotenv; load_dotenv(); import os;
print('OPENAI_API_KEY set:', bool(os.getenv('OPENAI_API_KEY')))"
```

Expected result: OPENAI_API_KEY set: True

10. Step F - Re-run Retrieval Evaluation

The retrieval evaluation was updated to use the same OpenAI embedding function. This was important because changing embeddings can change ranking and distance values.

```
uv run python -m backend.eval_retrieval
```

Observed final result:

```
=====
Retrieval Evaluation
```

```
=====
Hits: 10/10
Recall@3: 100.00%
```

This is a project evaluation over the 10 handcrafted ticket test cases. It demonstrates that the expected ticket appeared in the top three results for all 10 cases; it should not be described as a production benchmark.

11. Step G - Test the RAG Locally

Before deploying again, the application was checked with both providers and with an out-of-scope question. The out-of-scope question returned the fixed fallback: 'I don't know based on the available knowledge base. Please call 611 for assistance.'

This confirmed that retrieval and provider selection still worked after changing the embedding model.

12. Step H - Commit and Push the Memory Fix

The changed files were reviewed and staged:

```
git status
git add backend/embeddings.py backend/eval_retrieval.py backend/ingest_faq.py backend/
ingest_pdf.py backend/ingest_tickets.py backend/retriever.py pyproject.toml uv.lock
git status
```

Then the change was committed and pushed:

```
git commit -m "Use OpenAI embeddings to reduce backend memory"
git push
```

Commit created: 0ac0dd6.

13. Step I - Render Build Command Correction

The first post-migration Render build failed with `ModuleNotFoundError: No module named 'backend'`. The reason was the same import-context issue seen locally: the ingestion files now import `backend.embeddings` and therefore should be launched as modules.

The Render build command was changed to:

```
uv sync --frozen && uv run python -m backend.ingest_faq && uv run python -m
backend.ingest_tickets && uv run python -m backend.ingest_pdf
```

The Render start command remained:

```
uv run uvicorn backend.api:app --host 0.0.0.0 --port $PORT
```

Render then successfully built all three Chroma collections, uploaded the build, started Uvicorn, and reported that the service was live.

14. Step J - Render Environment Variables

The following variables are configured on Render. Their secret values must never be committed to GitHub or placed in the frontend.

Item	Status
GROQ_API_KEY	Secret - configured in Render
OPENAI_API_KEY	Secret - configured in Render
LLM_PROVIDER	Configured
FRONTEND_URL	https://telecom-rag-chatbot.vercel.app

FRONTEND_URL is used by FastAPI CORS configuration so the deployed Vercel website is permitted to call the Render backend.

15. Step K - Verify the Render Backend

The health endpoint was opened and returned:

```
{
  "status": "ok"
}
```

Streaming was tested directly from Terminal with Groq:

```
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"How can I activate international roaming?","provider":"groq"}'
```

The same endpoint was tested with OpenAI:

```
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"How can I activate international roaming?","provider":"openai"}'
```

The out-of-scope protection was tested with:

```
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"Who is the president of France?","provider":"groq"}'
```

Expected/observed fallback: I don't know based on the available knowledge base. Please call 611 for assistance.

16. Step L - Create the Vercel Account and Secure It

A Vercel Hobby workspace named 'Usha AI Projects' was used. The Vercel GitHub App was installed for the GitHub account so Vercel could import the repository.

Two-factor authentication was enabled using an authenticator app (TOTP). QR codes, TOTP codes, setup secrets, recovery codes, and API keys should always remain private.

17. Step M - Import Only the Frontend into Vercel

The GitHub repository contains both backend and frontend applications. In Vercel, the frontend was imported as a single project rather than deploying the FastAPI backend there.

Item	Status
Repository	ujoy100/telecom-rag-chatbot
Root Directory	frontend
Application Preset	Vite
Project	telecom-rag-chatbot
Plan	Hobby

18. Step N - Configure the Frontend Backend URL

The React app reads VITE_API_BASE_URL. This allows local development to use localhost while production uses the Render URL.

```
const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL || "http://127.0.0.1:8000";
```

In Vercel, the environment variable was configured as:

```
VITE_API_BASE_URL=https://telecom-rag-chatbot-poqa.onrender.com
```

No .env file was imported into Vercel. This is deliberate: the frontend only needs the public backend URL. OpenAI and Groq API keys belong only on the backend.

19. Step O - Deploy the Vercel Frontend

After the Vite preset, frontend root directory, and VITE_API_BASE_URL were configured, Deploy was selected. Vercel completed the build and created the stable production domain:

<https://telecom-rag-chatbot.vercel.app>

20. Step P - Connect Vercel to Render with CORS

The frontend and backend are on different domains, so the browser applies Cross-Origin Resource Sharing (CORS) rules. The Vercel domain therefore had to be explicitly allowed by the FastAPI backend.

On Render, the following variable was added:

```
FRONTEND_URL=https://telecom-rag-chatbot.vercel.app
```

Then 'Save, rebuild, and deploy' was selected. Render reported 'Deploy succeeded | Live'.

21. Step Q - Final End-to-End Browser Tests

The final tests were performed from the actual Vercel website, not localhost. This validates the full path: browser -> Vercel -> Render -> embeddings/retrieval -> selected LLM -> streamed answer -> browser.

Item	Status
Groq: international roaming question	PASS - Source FAQ 9

OpenAI: international roaming question	PASS - Source FAQ 9
Out-of-scope question	PASS - fixed 611 fallback
CORS	PASS - no browser CORS failure

22. Complete Command Reference

Run project commands from the repository root unless a command explicitly says otherwise.

```
# Check location
pwd

# Remove heavy dependency (performed during optimization)
uv remove torchvision

# Search for old embedding implementation
grep -RniE "HuggingFaceEmbeddings|langchain_huggingface|all-MiniLM|sentence-transformers"
backend --include="*.py"

# Safely verify local OpenAI key is available
uv run python -c "from dotenv import load_dotenv; load_dotenv(); import os;
print('OPENAI_API_KEY set:', bool(os.getenv('OPENAI_API_KEY')))"

# Rebuild knowledge base with OpenAI embeddings
uv run python -m backend.ingest_faq
uv run python -m backend.ingest_tickets
uv run python -m backend.ingest_pdf

# Evaluate retrieval
uv run python -m backend.eval_retrieval

# Run backend locally
uv run uvicorn backend.api:app

# Git review
git status
git diff --stat
git diff

# Stage deployment optimization
git add backend/embeddings.py backend/eval_retrieval.py backend/ingest_faq.py backend/
ingest_pdf.py backend/ingest_tickets.py backend/retriever.py pyproject.toml uv.lock

# Commit and push
git commit -m "Use OpenAI embeddings to reduce backend memory"
git push

# Render build command
```

```

uv sync --frozen && uv run python -m backend.ingest_faq && uv run python -m
backend.ingest_tickets && uv run python -m backend.ingest_pdf

# Render start command
uv run uvicorn backend.api:app --host 0.0.0.0 --port $PORT

# Production Groq API test
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"How can I activate international roaming?","provider":"groq"}'

# Production OpenAI API test
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"How can I activate international roaming?","provider":"openai"}'

# Production fallback test
curl -N -X POST https://telecom-rag-chatbot-poqa.onrender.com/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"Who is the president of France?","provider":"groq"}'

```

23. Common Problems We Encountered and What They Mean

Problem	Meaning	Fix
can't open ../backend/backend/ingest_faq.py	Command was run from backend while also specifying backend/ in the path.	Return to repository root, or use the correct relative path.
ModuleNotFoundError: No module named 'backend'	A script importing backend.* was launched as a file instead of as a package module.	From project root use: uv run python -m backend.ingest_faq (and equivalent commands).
OpenAIError: Missing credentials	OPENAI_API_KEY was not loaded into the process.	Load .env locally or configure OPENAI_API_KEY in the deployment environment. Never print the key.
Render out of memory >512 MB	Local SentenceTransformer/PyTorch startup exceeded free-instance RAM.	Use API embeddings and remove heavy local embedding dependencies.
HEAD / or GET / returns 404	No route was defined for the backend root URL.	Not a deployment failure. Use /health or /docs, or add a root route if desired.
First request is slow	Render free service may have spun down.	Wait for the instance to wake; later requests should be faster.
CORS error in browser	Backend does not allow the frontend origin.	Set FRONTEND_URL to the exact Vercel origin and redeploy Render.

24. Deployment Concepts for a Beginner

Build vs Start

Build happens before the web service starts. In this project, Render installs dependencies and creates the Chroma collections during the build. Start launches Uvicorn and keeps FastAPI running so it can receive web requests.

Environment Variables

Environment variables are configuration values supplied by the hosting platform. They keep secrets and environment-specific settings out of source code. `OPENAI_API_KEY` and `GROQ_API_KEY` are secrets; `VITE_API_BASE_URL` and `FRONTEND_URL` are configuration URLs.

CORS

CORS is a browser security mechanism. Because the Vercel frontend and Render backend have different origins, FastAPI must explicitly permit the Vercel origin.

Why `VITE_API_BASE_URL` starts with `VITE_`

Vite exposes selected build-time variables to browser code when their names start with `VITE_`. Therefore only public configuration, such as the backend URL, should use this mechanism. API secrets must never be placed in a `VITE_` variable.

Why `-m` matters

`python -m backend.ingest_faq` asks Python to run the file as part of the backend package. That makes imports such as `from backend.embeddings import get_embeddings` resolve correctly from the project root.

25. Security Rules to Keep

Never commit `.env`. Never put `GROQ_API_KEY` or `OPENAI_API_KEY` in Vercel frontend variables. Never share authenticator QR codes, TOTP codes, setup secrets, or recovery codes. Keep GitHub, Vercel, and Render accounts protected with 2FA. The public frontend URL and public Render service URL are safe to document; API keys are not.

26. Final Deployment Checklist

Item	Status
GitHub repository connected	DONE
Render backend build	DONE
Render backend live	DONE
OOM issue resolved	DONE
OpenAI embeddings	DONE
Chroma rebuilt	DONE
Recall@3 10/10	DONE
Render health test	DONE
Vercel frontend deployed	DONE
Vercel 2FA enabled	DONE
VITE_API_BASE_URL configured	DONE
Render FRONTEND_URL configured	DONE
Groq live browser test	DONE
OpenAI live browser test	DONE
Out-of-scope fallback test	DONE
Production deployment	COMPLETE

27. What to Update in the README

The repository README should now describe OpenAI text-embedding-3-small rather than the old HuggingFace MiniLM embedding model. It should include the live Vercel demo, Render backend/health URL, Render and Vercel deployment architecture, the 10/10 Recall@3 evaluation, the fallback behavior, and a short note that API embeddings were chosen to keep the Render service within the free-instance memory limit.

End of deployment guide